



Knowledge Transfer Workshop

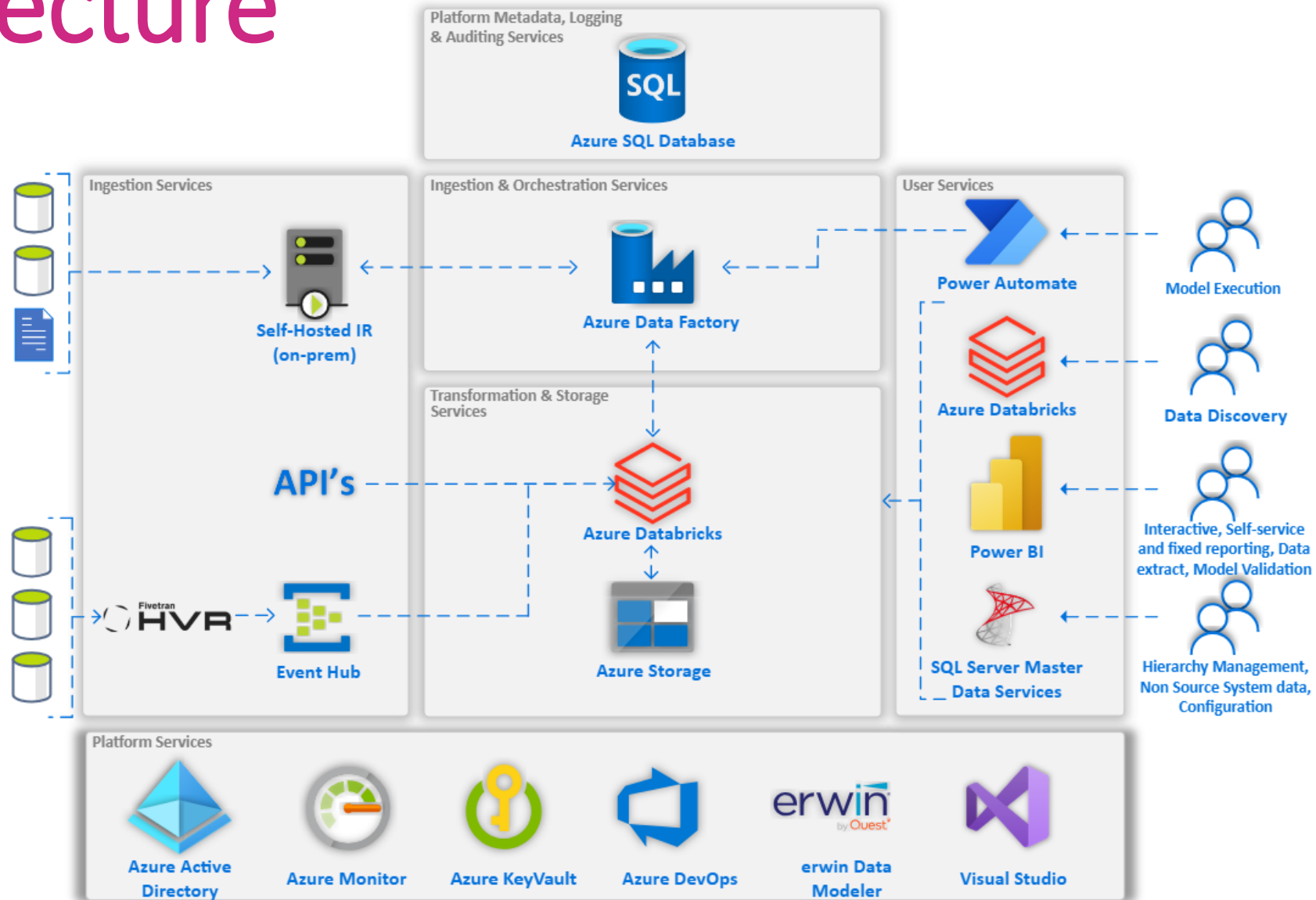
Overview

09/09/2024

Agenda

1. Architecture
2. Data Source
3. Data Flow & Layers
 - i. Source to Raw DLT / Source to Raw
 - ii. Raw to Standardised (includes ODA)
 - iii. Standardised to Conformed (includes BED)
 - iv. Conformed to DW (Dim -> Bridge -> Fact)
4. Metadata Usage and Task Overlap
5. Metadata Tables (Config & Audit)
6. Load Schedules
7. Orchestration
8. Reconciliation
9. Pull Requests & Release Pipelines
10. Catalogs and Squads

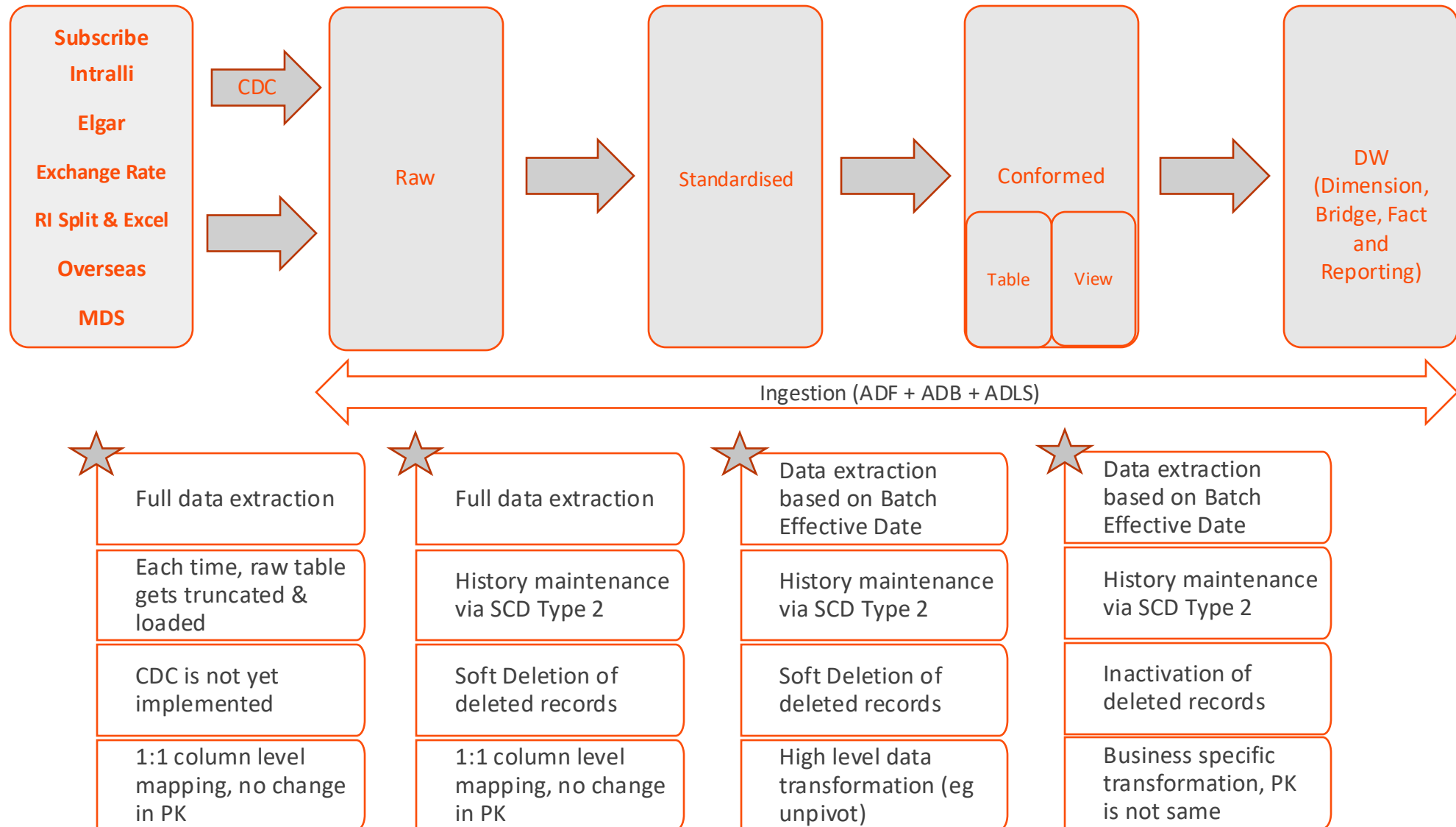
Architecture



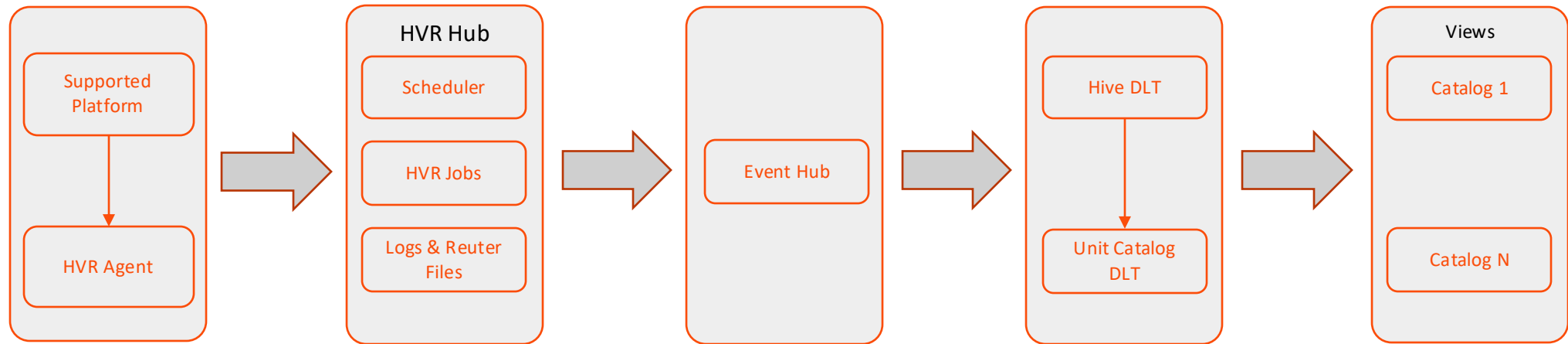
Data Sources

Source Name	Source Platform	Frequency	Subject Area
Subscribe	Database (HVR)	Twice a day	London Inwards Policy
Elgar	Database (HVR)	Twice a day	Outwards Policy
RI (Split & Spreadsheet)	Flat Files	Adhoc	Re-Insurance
FX (Oanda API & Files)	API & Flat Files	Daily & Adhoc	Exchange Rate
MDS	Database	Daily	Master Data Service
Bordereaux (Intrali & Viper)	Database	Daily	Bordereaux
Overseas	Database	Daily	Inwards Policy (OVerseas)

Data Flow

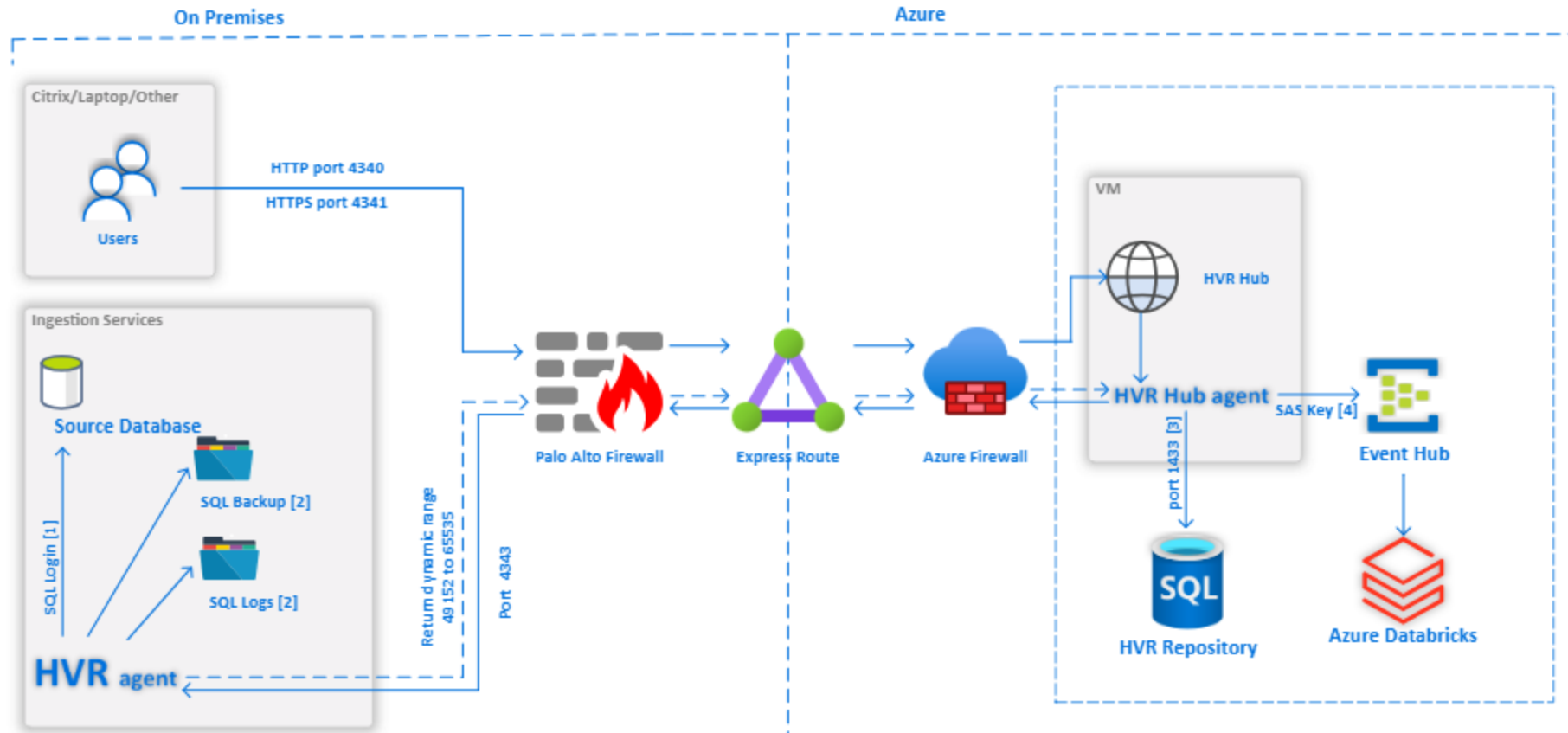


Source To Raw DLT (CDC using HVR)

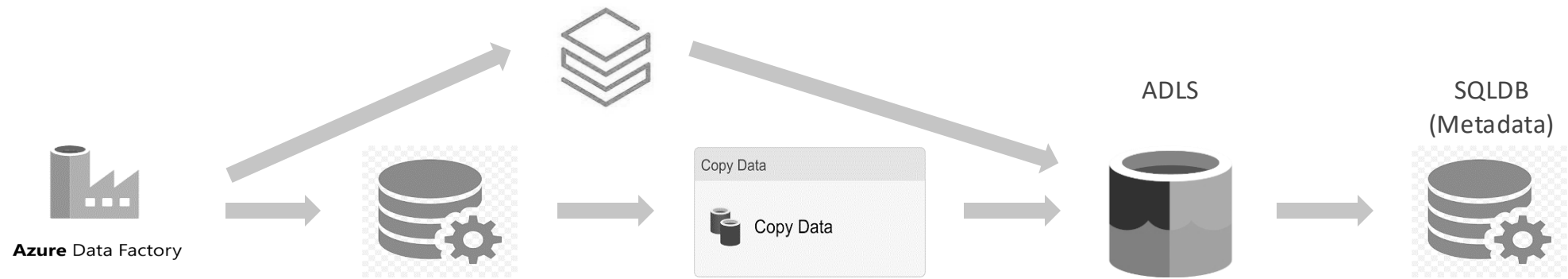


For more information, please refer to excel document - [CDC_Details.xlsx](#)

Source To Raw DLT (CDC using HVR)



Source To Raw (Non HVR)



Looks for active
Batch in SQLDB

Calls the procedure
usp_get_object_con
nectionStringDetails

This procedure
generates a
where_clause



Truncate the table
in raw



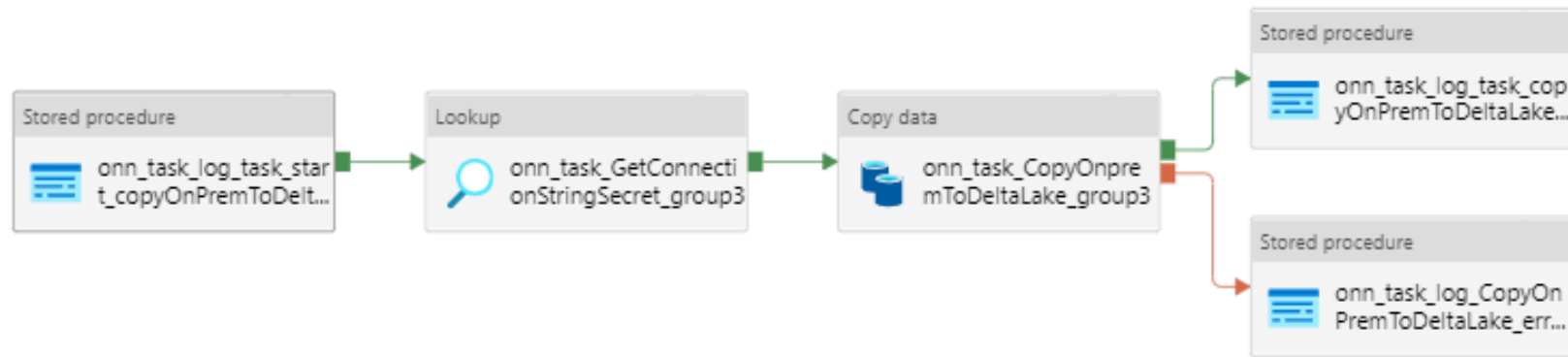
Loads the data in
delta table



Update the object
date availability for
raw tables

Update the log
tables

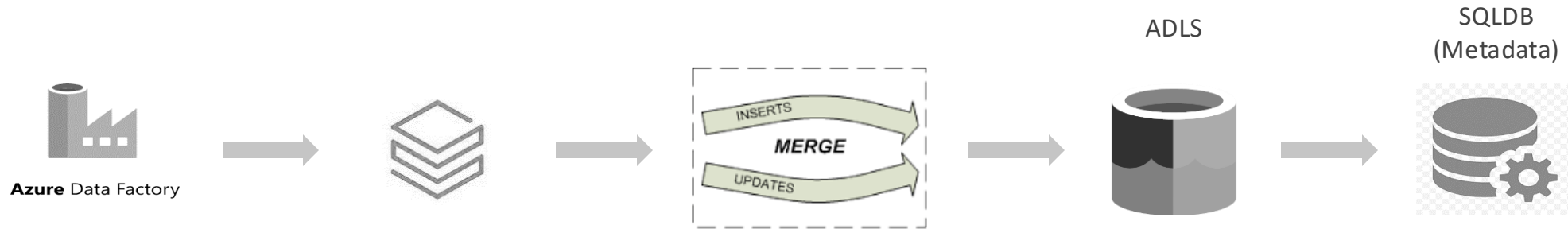
Source To Non HVR Raw



Summary:

- “onn_task_GetConnectionStringSecret_grou” Activity – calls SQLDB procedure, usp_get_object_connectionStringDetails that helps generates a connection string in order to connect to the on premises source. It also updates where clause to “Full_Refresh”
- “onn_task_CopyOnpremmToDeltaLake_group3” Activity – Establishes connecting using connecting string, truncates the target table and loads data in raw table
- “onn_task_log_task_copyOnPremToDeltaLakeend_group” Activity – calls SQLDB procedure, usp_log_sql_task_end which further calls procedure, usp_mark_dates_loaded. This activity thus helps to update tbl_object_dates_availability to mark a batch_task_id for an object (raw) as loaded

Raw To Standardised



Looks for active Batch in SQLDB

Calls notebook load_scd2withdelete_dbdt_dbdt

Update the log tables



Fetches data based on entries in object dates availability

This helps to filter only for new data from Raw

Apply the merge statement - insert, update & soft delete



Loads the data in delta table



Update the object date availability for standardised tables

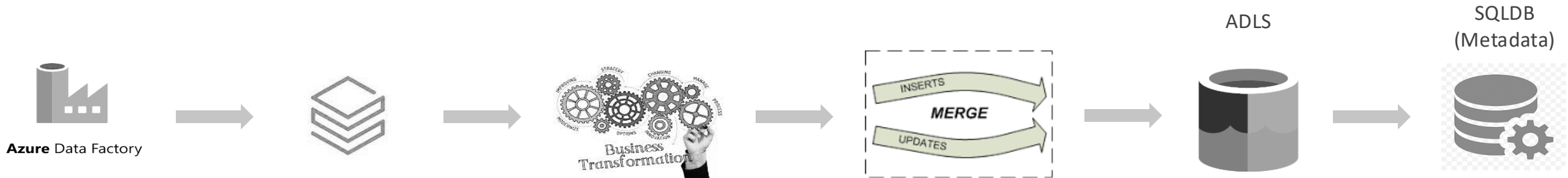
Update the log tables

Incremental Filter – Raw & Standardised

Batch Task Id	Input Object	Output Object	Input DateTime From	Input DateTime To	Output DateTime From	Output DateTime To
1001	dbo.PolMain	Raw.dbo_PolMain	2000-01-01 00:00:01.000	2022-04-29 09:09:34.500	2000-01-01 00:00:01.000	2022-04-29 09:09:34.500
1002	Raw.dbo_PolMain	Std.dbo_PolMain	2000-01-01 00:00:01.000	2022-04-29 09:09:34.500	2000-01-01 00:00:01.000	2022-05-09 16:19:31.043
2001	dbo.PolMain	Raw.dbo_PolMain	2022-04-29 09:09:34.500	2022-05-10 15:10:04.987	2022-04-29 09:09:34.500	2022-05-10 15:10:04.987
2002	Raw.dbo_PolMain	Std.dbo_PolMain	2022-04-29 09:09:34.500	2022-05-10 15:10:04.987	2022-05-12 08:40:01.282	2022-05-12 08:40:01.282

To understand the oda population of conformed, please refer to the excel document - [Example-Object_Dates_Availability.xlsx](#)

Standardised To Conformed



Looks for active Batch and BED in SQLDB

Calls custom notebook for individual entity

Update the log tables



Fetches data based on entries in object dates availability

This helps to filter only for new data from standardised

Custom SQL helps to fetch data as per target structure

BED filter is applied in custom SQL Notebook

NULLs are assigned to respective DEFAULT values



Apply the merge statement - insert, update & soft delete

Merge joins are done on Partition Key and HBK



Loads the data in delta table



Update the object date availability for conformed tables

Update the log tables

Raw To Standardised (Merge)

Sr.	Scenario	Outcome	Description
1	A new record comes from Raw table that does not exist in standardised Eg: HBK = 1, HV = 1, LCT = D1	Record gets inserted as new and active record in standardised Eg: HBK = HBK1, HV = HV1, VF = D1, VT = NULL, DT = NULL	New record get inserted
2	A record (existing in Standardised) comes from Raw with some updates Eg: HBK = 1, HV = 2, LCT = D2	Existing active record marked as inactive Record gets inserted as new and active record in standardised Eg: HBK = HBK1, HV = HV1, VF = D1, <u>VT = D2</u> , NULL, DT = NULL HBK = HBK1, <u>HV = 2</u> , <u>VF = D2</u> , VT = NULL, DT = NULL	Existing record closed, updated record inserted
3	A record (existing in Standardised) comes from Raw with no updates Eg: HBK = 1, HV = 2, LCT = D3	No change in Standardised table Eg: HBK = HBK1, HV = HV1, VF = D1, VT = D2, NULL, DT = NULL HBK = HBK1, HV = HV2, VF = D2, VT = NULL, DT = NULL	No impact on existing dataset
4	A record (existing in Standardised) doesn't appear in Raw	Existing Record gets updated with DTS in standardised Eg: HBK = HBK1, HV = HV1, VF = D1, VT = D2, NULL, DT = NULL HBK = HBK1, HV = HV2, VF = D2, VT = NULL, <u>DT = D4</u>	Existing active record marked as deleted
5	Same record (mentioned step 4) re-appears in Raw Eg: HBK = 1, HV = 2, LCT = D5	Existing active record marked as inactive Record gets inserted as new and active record in standardised Eg: HBK = HBK1, HV = HV1, VF = D1, VT = D2, NULL, DT = NULL HBK = HBK1, HV = HV2, VF = D2, <u>VT = D5</u> , DT = D4 HBK = HBK1, HV = HV2, <u>VF = D5</u> , VT = NULL, DT = NULL	Existing active record marked as inactive, new active record inserted

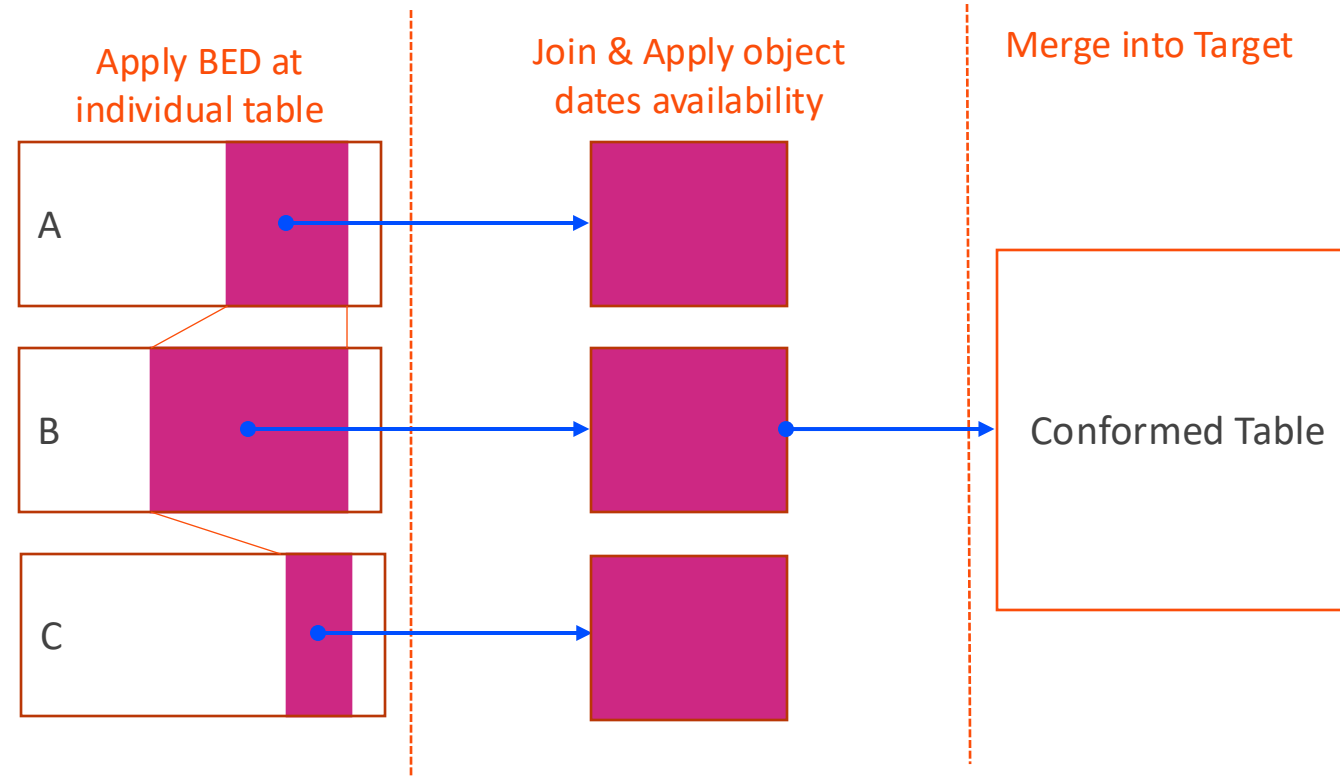
Standardised To Conformed (Merge)

Sr.	Scenario	Outcome	Description
1	A new record comes from Standardised table Eg: HBK = 1, HV = 1, VFT = D1, VTT = NULL, DT = NULL	Record gets inserted as new and active record in conformed Eg: HBK = 1, HV = 1, VFT = D1, VTT = NULL, DT = NULL	New record get inserted
2	A record (existing conformed) comes from standardised with some updates as active Eg: HBK = 1, HV = 2, VFT = D2, VTT = NULL, DT = NULL	Existing active record marked as inactive Record gets inserted as new and active record in conformed Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2 , NULL, DT = NULL HBK = 1, HV = 2, VFT = D2 , VTT = NULL, DT = NULL	Existing record closed, updated record inserted as active
3	A record (existing in conformed as active) comes from standardised with some updates as well as lakeDeletedTimestamp Eg: HBK = 1, HV = 3, VFT = D3, VTT = NULL, DT = D3	Existing active record marked as inactive Record gets inserted as deleted and active record in conformed Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2, NULL, DT = NULL HBK = 1, HV = 2, VFT = D2, VTT = D3 , DT = NULL HBK = 1, HV = 3, VFT = D3 , VTT = NULL, DT = D3	Existing record closed, updated record inserted as deleted
4	A record (existing in conformed as deleted) comes from standardised with some updates as well as lakeDeletedTimestamp Eg: HBK = 1, HV = 4, VFT = D4, VTT = NULL, DT = D4	Existing active record marked as inactive Record gets inserted as deleted and active record in conformed Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2, NULL, DT = NULL HBK = 1, HV = 2, VFT = D2, VTT = D3, DT = NULL HBK = 1, HV = 3, VFT = D3, VTT = D4 , DT = D3 HBK = 1, HV = 4, VFT = D4 , VTT = NULL, DT = D4	Existing record closed, updated record inserted as deleted
5	A record (existing in conformed as deleted) comes from standardised with some update and as active Eg: HBK = 1, HV = 5, VFT = D5, VTT = NULL, DT = NULL	Existing active record marked as inactive Record gets inserted as new and active record in conformed Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2, NULL, DT = NULL HBK = 1, HV = 2, VFT = D2, VTT = D3, DT = NULL HBK = 1, HV = 3, VFT = D3, VTT = D4, DT = D3 HBK = 1, HV = 4, VFT = D4, VTT = D5 , DT = D4 HBK = 1, HV = 5, VFT = D5 , VTT = NULL, DT = NULL	Existing record closed, updated record inserted as active

Standardised To Conformed (Merge)

Sr.	Scenario	Outcome	Description
6	A record (existing in conformed as active) comes from standardised with no change and as active Eg: HBK = 1, HV = 5, VFT = D5, VTT = NULL, DT = NULL	No change in conformed table Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2, NULL, DT = NULL HBK = 1, HV = 2, VFT = D2, VTT = D3, DT = NULL HBK = 1, HV = 3, VFT = D3, VTT = D4, DT = D3 HBK = 1, HV = 4, VFT = D4, VTT = D5, DT = D4 HBK = 1, HV = 5, VFT = D5, VTT = NULL, DT = NULL	No impact on existing dataset
7	A record (existing in conformed as active) comes from standardised with no change but with some lakeDeletedTimestamp Eg: HBK = 1, HV = 5, VFT = D5, VTT = NULL, DT = D7	Existing active record marked as inactive Record gets inserted as deleted and active record Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2, NULL, DT = NULL HBK = 1, HV = 2, VFT = D2, VTT = D3, DT = NULL HBK = 1, HV = 3, VFT = D3, VTT = D4, DT = D3 HBK = 1, HV = 4, VFT = D4, VTT = D5, DT = D4 HBK = 1, HV = 5, VFT = D5, VTT = NULL, <u>DT = D7</u>	Existing record closed, updated record inserted as deleted
8	A record (existing in conformed as deleted) comes from standardised with no change but with some lakeDeletedTimestamp Eg: HBK = 1, HV = 5, VFT = D8, VTT = NULL, DT = D8	No change in conformed table Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2, NULL, DT = NULL HBK = 1, HV = 2, VFT = D2, VTT = D3, DT = NULL HBK = 1, HV = 3, VFT = D3, VTT = D4, DT = D3 HBK = 1, HV = 4, VFT = D4, VTT = D5, DT = D4 HBK = 1, HV = 5, VFT = D5, VTT = NULL, DT = D7	No impact on existing dataset
9	A record (existing in conformed as deleted) comes from standardised with no change and as active Eg: HBK = 1, HV = 5, VFT = D9, VTT = NULL, DT = NULL	Existing active record marked as inactive Record gets inserted as active record in conformed Eg: HBK = 1, HV = 1, VFT = D1, VTT = D2, NULL, DT = NULL HBK = 1, HV = 2, VFT = D2, VTT = D3, DT = NULL HBK = 1, HV = 3, VFT = D3, VTT = D4, DT = D3 HBK = 1, HV = 4, VFT = D4, VTT = D5, DT = D4 HBK = 1, HV = 5, VFT = D5, <u>VTT = D9</u>, DT = D7 HBK = 1, HV = 5, <u>VFT = D9</u>, VTT = NULL, DT = NULL	Existing record closed, updated record inserted as active

Incremental Filter – Conformed

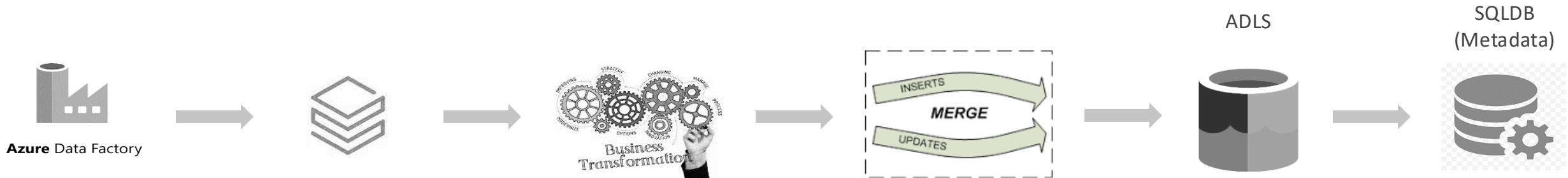


When we fetch the data from multiple standardised tables, filter of BED is applied on each table. Thus, the data gets restricted in the custom SQL

Based on the source phase dates availability populated for tables A, B & C in standardised layer. A where_clause is generated on partition column taking min & max date of data loaded across three tables.

The source data gets filtered based on where_clause post which it gets inserted, updated or soft deleted in target conformed table

Conformed to DW



Looks for active Batch and BED in SQLDB

Calls custom notebook for individual entity

Update the log tables



Fetches data based on entries in object dates availability

This helps to filter only for new data from conformed

Custom SQL helps to fetch data as per target structure

BED filter is applied in custom SQL Notebook

NULLs are assigned to respective DEFAULT values



Apply the merge statement - insert, update & soft delete

Append Contra and Deletes in Fact



Loads the data in delta table



Update the object date availability for conformed tables

Update the log tables

Metadata usage and task overlap

	Raw	Standardised	Conformed Table	Conformed View	Dimension/Fact Table
Description	Source specific physical table in databricks holding latest extract from source	Source specific physical table in databricks holding all type 2 historical data	Source specific physical table in databricks holding all type 2 historical data that has been conformed across multiple standardised tables	View sitting over source specific conformed tables –should be identical structure to warehouse staging table and warehouse external view	
Modelled in ERWIN	Yes	Yes	Yes	No	Yes
Creates HBK	No	Yes	Yes	No	Yes
Metadata Object defined (table/view)	Yes	Yes	Yes	Yes	No
Metadata Object definition defined (columns)	Yes	Yes	Yes	No	No
Metadata Object Primary Key	No	Yes	Yes	No	No
Metadata column to column level mapping	Yes	Yes	No	No	No
Metadata type 2 columns defined	No	Yes	Yes	No	No
Metadata task	Yes	Yes	Yes	No	Yes

Metadata Tables (config)

Schema	Table Name	Description
Config	Phase	Phases across which data is being stored eg: Raw, Standardised, Conformed, Dwh (Dim, Bridge &Fact) and Reporting
Config	Source	Sources from which data is being fetched into insights
Config	Schedule	Timing at which various source gets executed across multiple phases
Config	Entity	
Config	Schedule_Phase_Entity_Map	Entities that gets loaded when a schedule is triggered across multiple phases
Config	Databricks_Workspace	Databricks workspaces details such as URL, workspace name, resource id and keyvault reference
Config	Databricks_Cluster	Clusters associated with individual catalog workspace
Config	Databricks_Cluster_Schedule	Clusters associated with individual schedules when it gets triggered
Config	Activity	All the notebooks present in insights with location at which it is present
Config	Task	
Config	Task_Type	
Config	Task_Object_Map	
Config	Object	
Config	Object_Definition	
Config	Object_To_Object_Map	
Config	Source_Phase_Treshold	

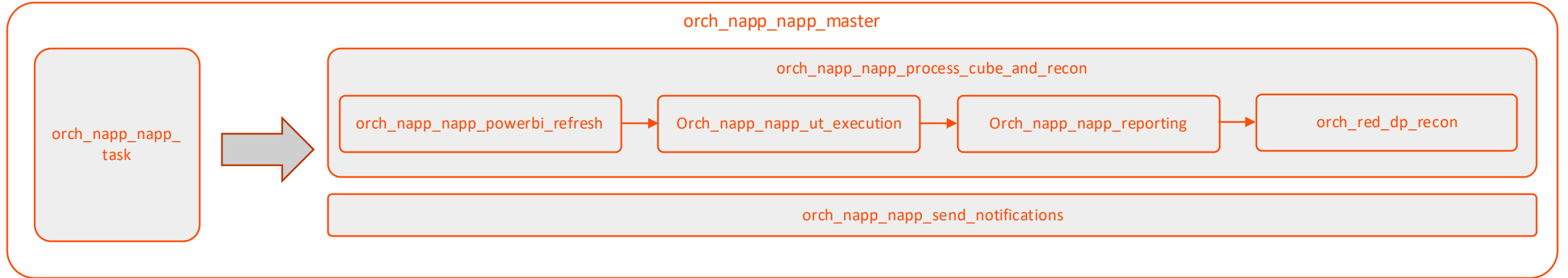
Metadata Tables (audit)

Schema	Table Name	Description
Audit	Batch	
Audit	Batch_Task	
Audit	Error_Log	
Audit	Object_Dates_Availability	
Audit	Source_Phase_Dates_Availability	

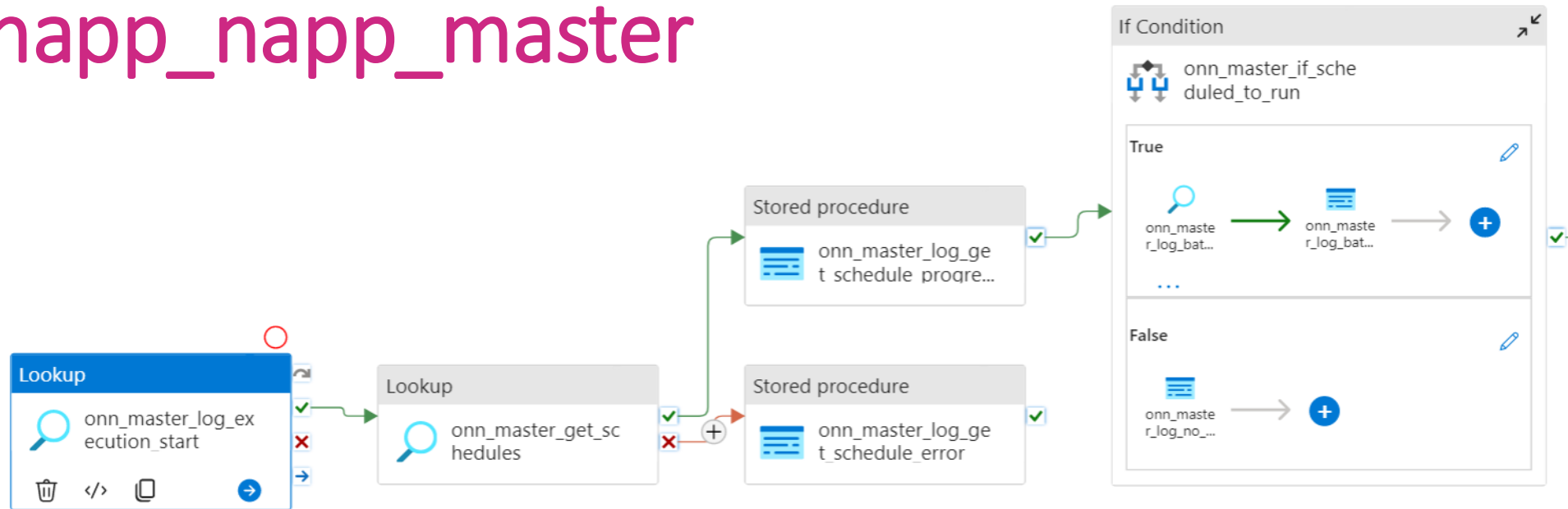
Load Schedules

Source Name	Layer / Process	Day1																			Day2																	
	B(D)ST Timing --->	06:00	07:00	08:00	09:00	10:00	11:00	12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00	21:00	22:00	23:00	00:00	01:00	02:00	03:00	04:00	05:00	06:00												
HVR Hub VM (Test)	UTC/GMT Timing --->																																					
HVR Hub VM (Prod)	UTC/GMT Timing --->																																					
Subscribe	App db to RED Restore													18:30	20:30																							
Elgar	App db to RED Restore													18:30	21:05																							
Overseas	App db to RED Restore																22:15																					
Intrali (UTC)	XX to SonarCube Restore													20:20																								
Exchange Rate	Oanda API																		00:15		02:15																	
RED (Global Load)	N/A																		23:00		01:59																	
RED Recon																					02:00																	
Subs & Elgar	Source to RawDLT								13:45									19:45																				
Subs & Elgar	RawDLT to Standardised										15:00									21:00																		
Overseas	Source to Standardised																		22:30		23:40																	
MDS	Source to Standardised																20:50		23:40																			
Intrali	Source To Standardised																21:00		23:40																			
Exchange Rate	Source to Standardised																					02:00																
All Sources	Conformed to Fact Batch																					23:40		03:50														
Power BI Refresh (All)																						00:30		01:30														
Recon & UT (All)																								02:00		03:45												
Exchange Rate	Conformed to Fact Batch																										04:30											
Power BI Refresh (FX)																											04:45											
Only UT (FX)																											04:30											

Orchestration Overview



orch_napp_napp_master

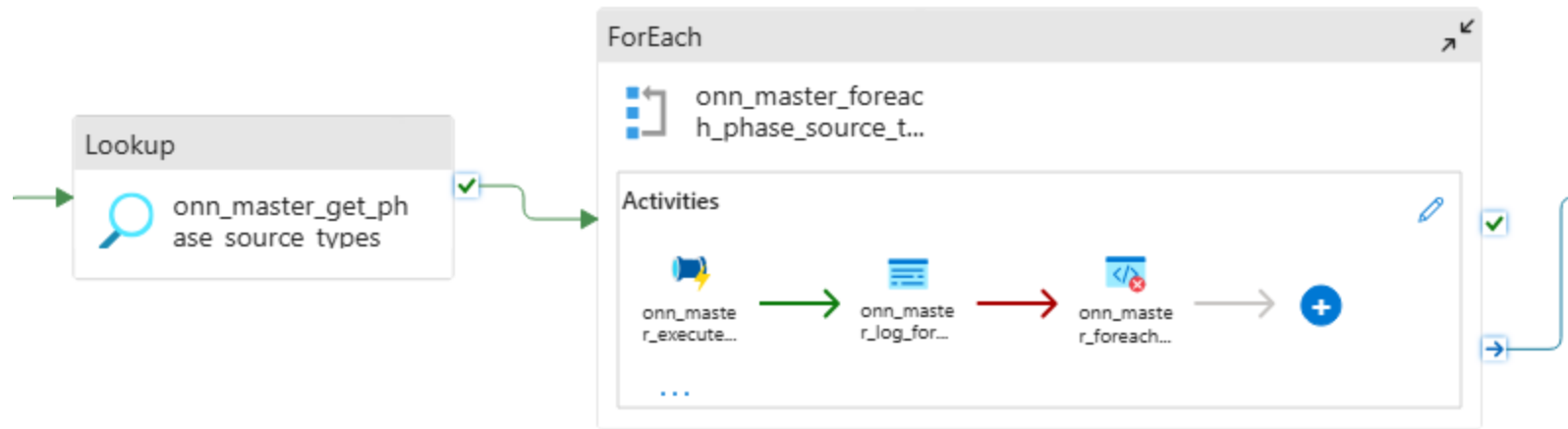


Step	Activity Name	Description
1	Lookup - onn_master_log_execution_start	- Executes audit.usp_log_adf_pipeline_execution_start - Populates audit table [audit].[tbl_log_adf_pipeline_execution] - Contains adf execution details - execution id, run id, start end time, status, batch id
2	Lookup - onn_master_get_schedules	- Executes config.usp_get_schedule_reference - Checks at a particular time, what all schedules are active (when the master pipeline is triggered) - For a particular adf_pipeline_schedule_group, if adf_pipeline_schedule_group=schedule_priority, those schedule ids will run in parallel, others will run post their completion - tbl_schedule is joined with tbl_batch to check if a batch is running for a schedule priority lower to the ones we would like to start - It returns 3 values -->schedule_reference, schedule_to_run, batch_is_running - If a batch is still running for a schedule priority lower to the ones we want to start, schedule reference will have value 'Batch is running', schedule to run will have value 0 and batch_is_running will have value 1 - For a batch to start, schedule to run should be 1 and batch is running should be 0, i.e., no batch for the lower priority should be in running state

orch_napp_napp_master

Step	Activity Name	Description
3	Stored Proc - onn_master_log_get_schedule _progress	• Executes audit.usp_log_adf_pipeline_progress • Inserts the value for pipeline_activity_duration(ss) column in log tbl audit.tbl_log_adf_pipeline_progress
4	Stored Proc - onn_master_log_get_schedule _error	• Executes audit.usp_log_error in case onn_master_get_schedules fails
5	If Condition - onn_master_if_scheduled_to_ run	• This checks if we have any schedule that can be triggered
5 (a)	Stored proc - onn_master_log_batch_start_ progress	• If the above condition is True ○ Executes audit.usp_batch_start by taking schedule reference as input ○ Populates table audit.tbl_batch and audit.tbl_batch_task
5 (b)	Stored Proc - onn_master_log_batch_start_ progress	• On Success of Activity (5a) ○ Executes [audit].[usp_log_adf_pipeline_progress]
5 (c)	Set Variable	• On Success of Activity (5a) ○ Stores the output of audit.usp_batch_start stored proc
5 (d)	Stored Proc - onn_master_log_batch_start_ error	• On Failure of Activity (5a) ○ Executes [audit].[usp_log_error] ○ Populates [audit].[tbl_error_log]
5 (e)	Stored Proc - onn_master_log_no_schedule _to_run	• If the above condition is False ○ Executes [audit].[usp_log_adf_pipeline_progress] ○ Populates pipeline progress message as no schedule to run

orch_napp_napp_master



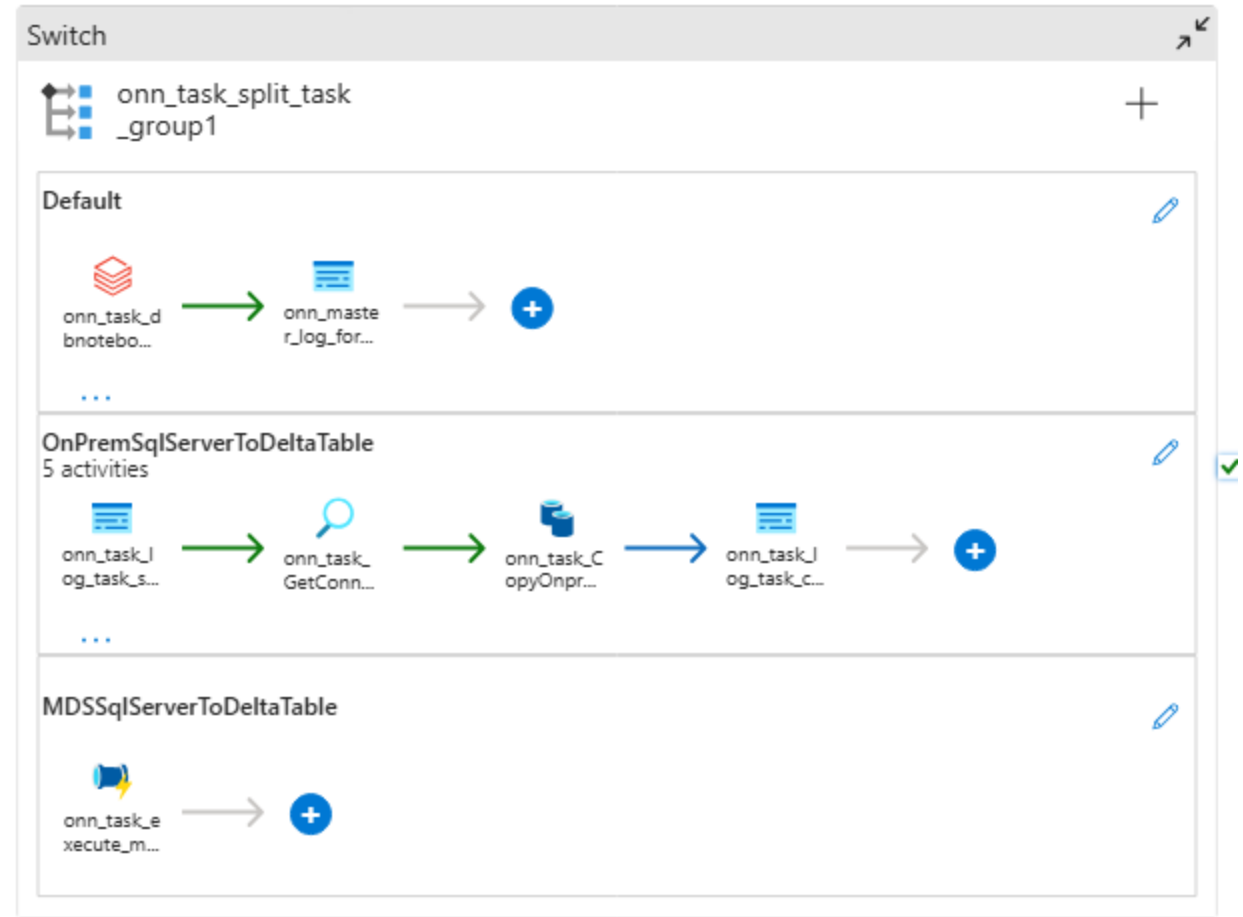
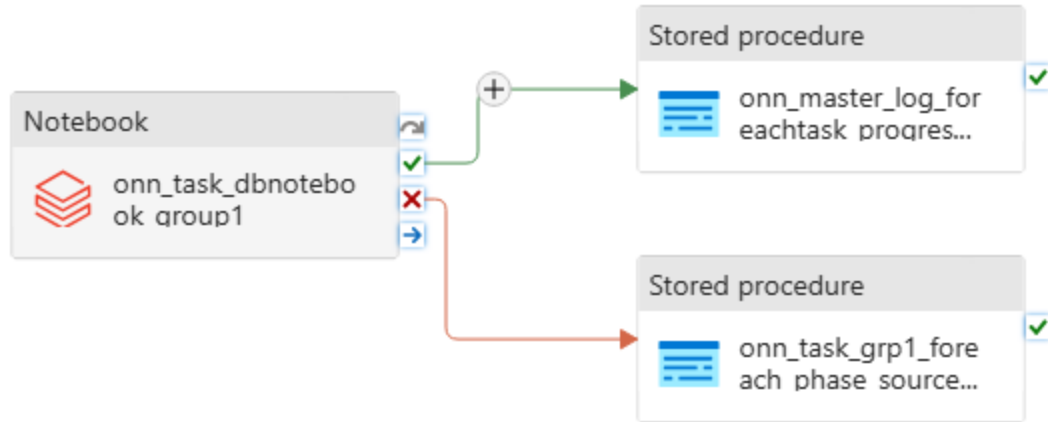
Step	Activity Name	Description																								
6	Lookup - onn_master_get_phase_source_t ypes	- Executes [config].[usp_get_phase_sourcetype_taskpriority] batch_id - This gives the phase_id, source_type_id, source_type_name, source_type_priority, task_priority, phase_priority, phase_name for a particular batch - Next activity will loop through the result set of the stored proc <table><tr><th></th><th>phase_id</th><th>source_type_id</th><th>source_type_name</th><th>source_type_priority</th><th>task_priority</th><th>phase_priority</th><th>phase_name</th></tr><tr><td>1</td><td>1</td><td>4</td><td>DataSource</td><td>2</td><td>1</td><td>10</td><td>raw</td></tr><tr><td>2</td><td>3</td><td>4</td><td>DataSource</td><td>2</td><td>1</td><td>30</td><td>standardised</td></tr></table>		phase_id	source_type_id	source_type_name	source_type_priority	task_priority	phase_priority	phase_name	1	1	4	DataSource	2	1	10	raw	2	3	4	DataSource	2	1	30	standardised
	phase_id	source_type_id	source_type_name	source_type_priority	task_priority	phase_priority	phase_name																			
1	1	4	DataSource	2	1	10	raw																			
2	3	4	DataSource	2	1	30	standardised																			
9	ForEach - onn_master_foreach_phase_sour ce_type	This takes the above result set and checks the phase name, by default it executes napp_napp_task																								

orch_napp_napp_task



Step	Activity Name	Description
8	Stored Proc - onn_task_prepare_task	Entry in source phase dates availability
9	Lookup – onn_task_get_task_group	Divides tasks into group of max 30 across three groups (overall 90)
10	For Each – onn_tasks_foreach_phase_source_type_task	Gets input from previous activity based on that performs either execution of notebook or copy activity

orch_napp_napp_task

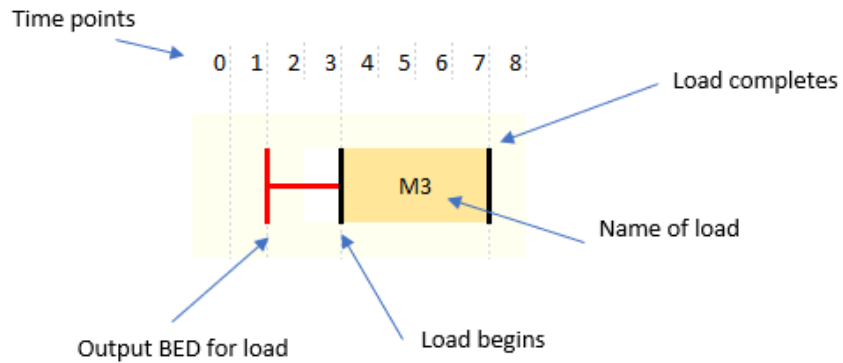


Step	Activity Name	Description
1	Notebook - onn_task_dbnotebook_group	• Calls custom notebook for individual entities • The notebook has set of commands that gets executed only when flag (generated by using history of transaction performed on delta table) dependentTablesUpdated = true i.e. if none of source table are update the commands won't get executed • The notebook further calls procedure, usp_mark_dates_loaded. This activity thus helps to update tbl_object_dates_availability to mark a batch_task_id for an object (conformed) as loaded

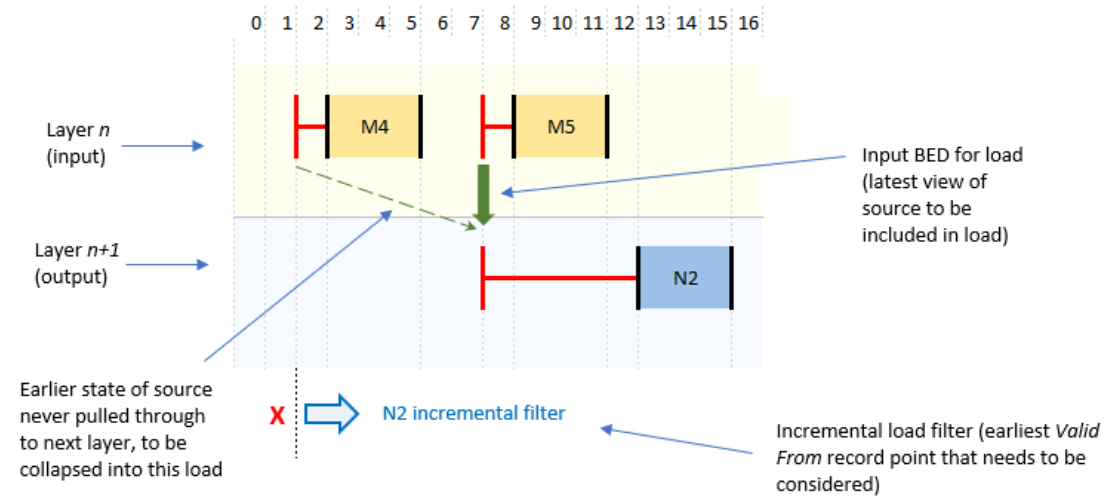
BED - Explanation of Diagram Notation

Specific notation is used in the following scenario diagrams to illustrate important aspects of the incremental loads

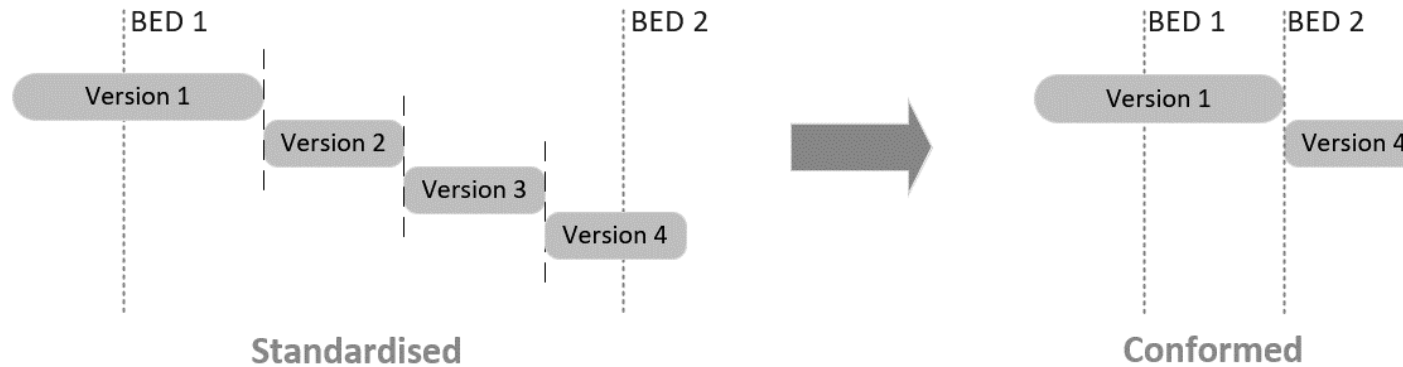
Basic Notation



Notation for Loading between Layers



Standardised To Conformed (BED)

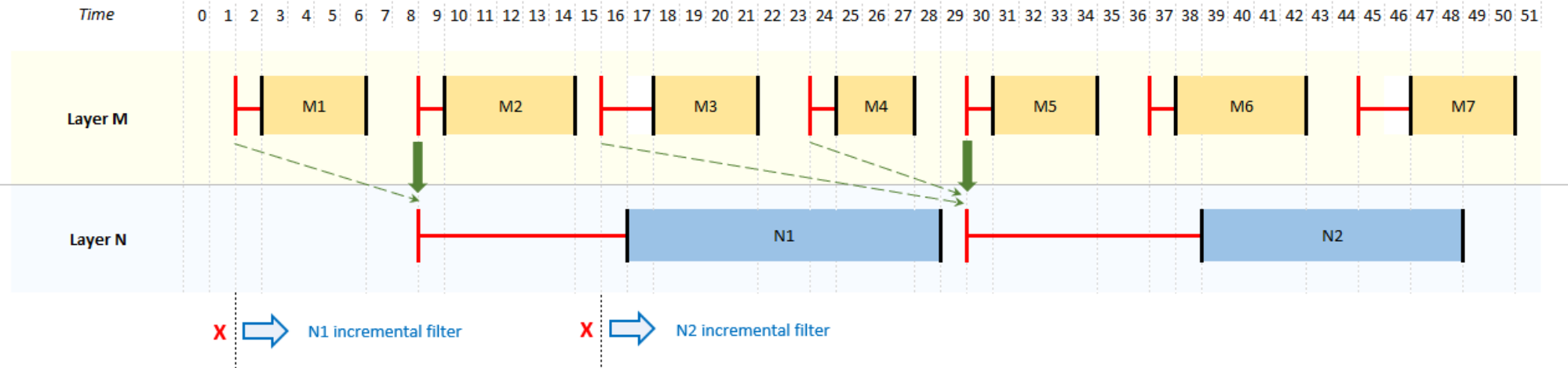


The essence of the BED approach can be captured as follows:

- Incremental loads will be conducted in batches during which a clearly-defined window of data will be processed to Conformed. There can be multiple such batches processed per day. Each batch has an associated “Batch Effective Date”, which is the latest time considered during processing of that batch
- Within a batch, if the same source record is updated multiple times, only the latest version of that record will be processed to Conformed
- All records created/updated/deleted in Conformed as a result of the batch will receive ValidFrom / ValidTo timestamps equal to the BED of that batch

BED - Single Source Load, Scenario 1

Output layer loaded using latest available consistent data from input layer

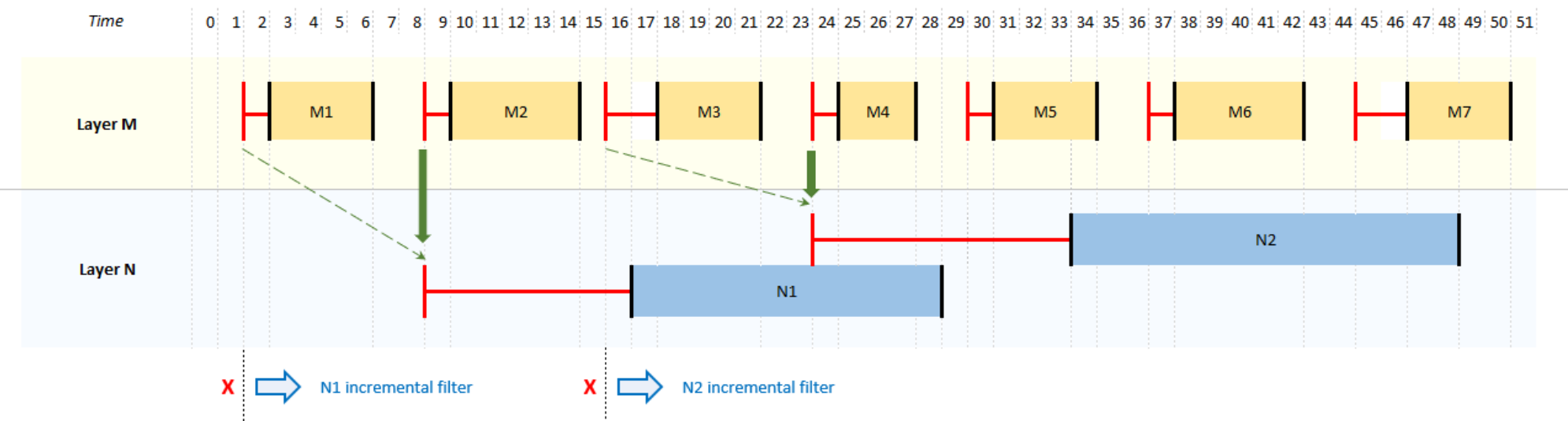


Load Metadata

Load	Input Info		Load Config		
	Input Loads	BEDs	Input BED	Incremental Filter	Output BED
N1	M1, M2	1, 8	8	1	8
N2	M3, M4, M5	15, 23, 29	29	15	29

BED - Single Source Load, Scenario 2

Output layer loaded using latest available consistent data from input layer



Load Metadata

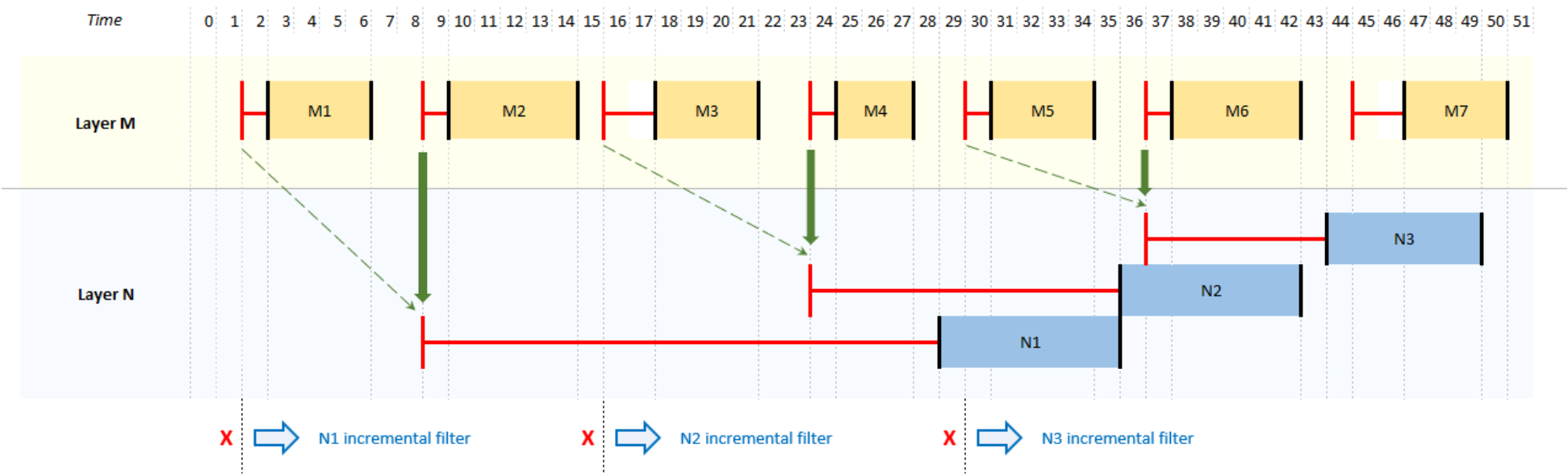
Load	Input Info		Load Config		
	Input Loads	BEDs	Input BED	Incremental Filter	Output BED
N1	M1, M2	1, 8	8	1	8
N2	M3, M4	15, 23	23	15	23

NOTES

BED for load N2 overlaps within execution window of load N1, but this poses no problem in practice (the loads themselves do not run in parallel)

BED - Single Source Load, Scenario 3

Output layer loaded using historical slices of data following a period without loads, to catch-up in small chunks rather than lumping all change into a small window



Load Metadata

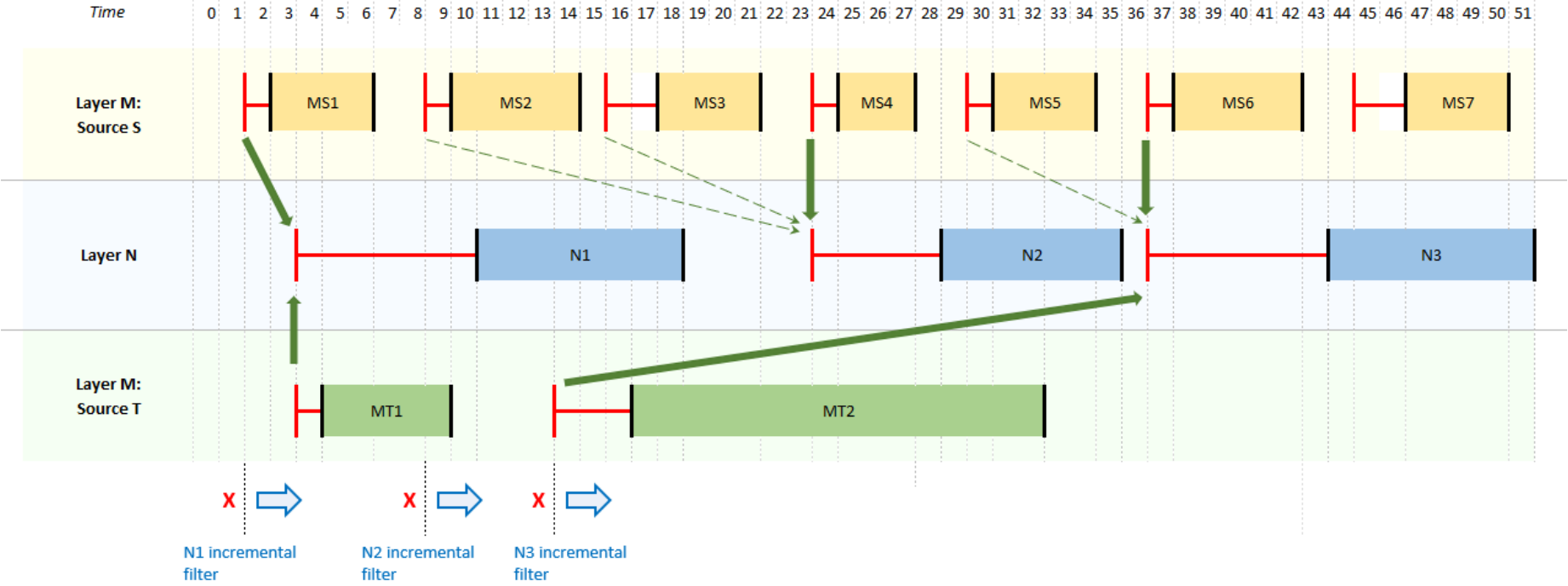
Load	Input Info		Load Config		
	Input Loads	BEDs	Input BED	Incremental Filter	Output BED
N1	M1, M2	1, 8	8	1	8
N2	M3, M4	15, 23	23	15	23
N3	M5, M6	29, 36	36	29	36

NOTES

Load N1 could have potentially pulled data from source loads M1, M2, M3 and M4 all together, as these had all completed prior to start of N1. However to avoid to large a quantum of change in a single layer N load, Input BED is chosen such that N1 only targets M1 and M2.

BED - Multiple Source Load, Scenario 1

Output layer loaded using latest available consistent data from input layer



Load Metadata

Load	Input Info			Load Config		
	Source	Input Loads	BEDs	Input BED	Incremental Filter	Output BED
N1	S	MS1	1	1	1	3
	T	MT1	3	3		
N2	S	MS2, MS3, MS4	8, 15, 23	23	8	23
	T	-	-	3		
N3	S	MS5, MS6	29, 36	36	13	36
	T	MT2	13	13		

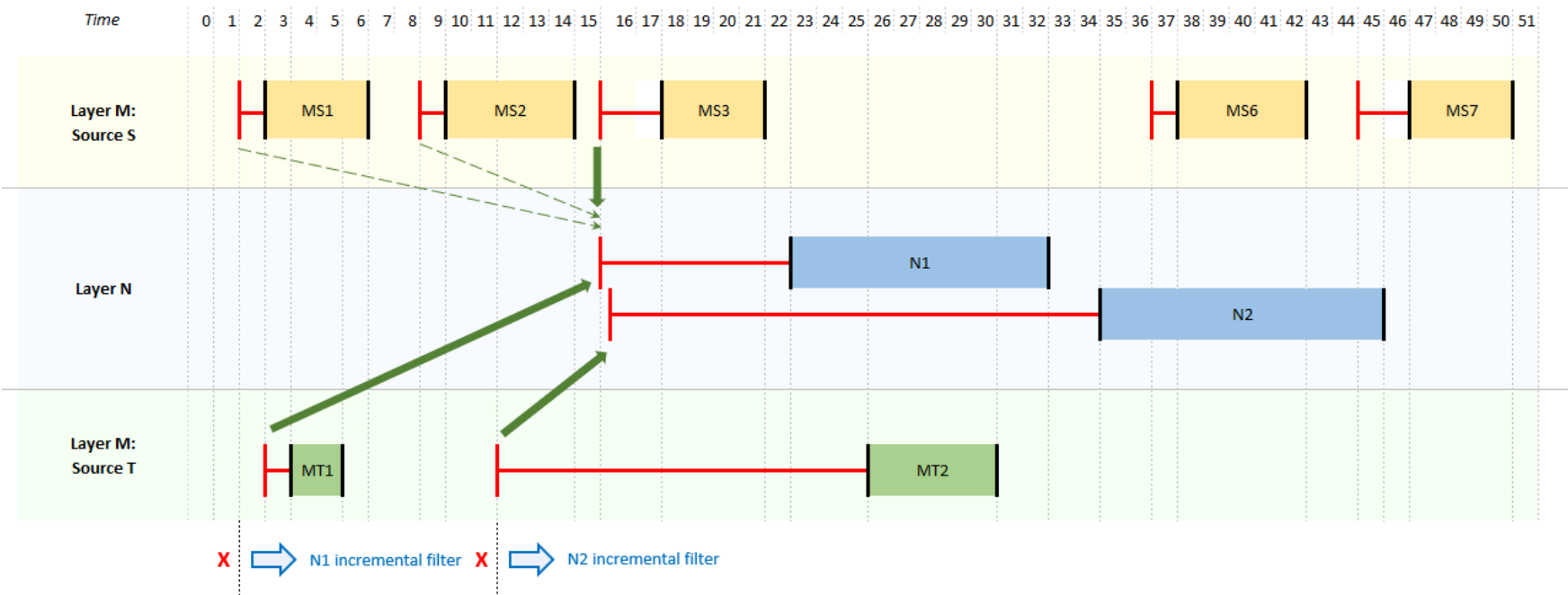
NOTES

Source T has not updated in-between higher-layer loads N1 and N2, so its Input BED is unchanged.

Incremental filter is now the minimum of the BEDs across all source loads in scope

BED - Multiple Source Load, Scenario 2

Output layer loaded using latest available consistent data from input layer



Load Metadata

Load	Input Info			Load Config		
	Source	Input Loads	BEDs	Input BED	Incremental Filter	Output BED
N1	S	MS1, MS2, MS3	1, 8, 15	15	1	15
	T	MT1	2	2		
N2	S	-	-	15	11	15+ε
	T	MT2	11	11		

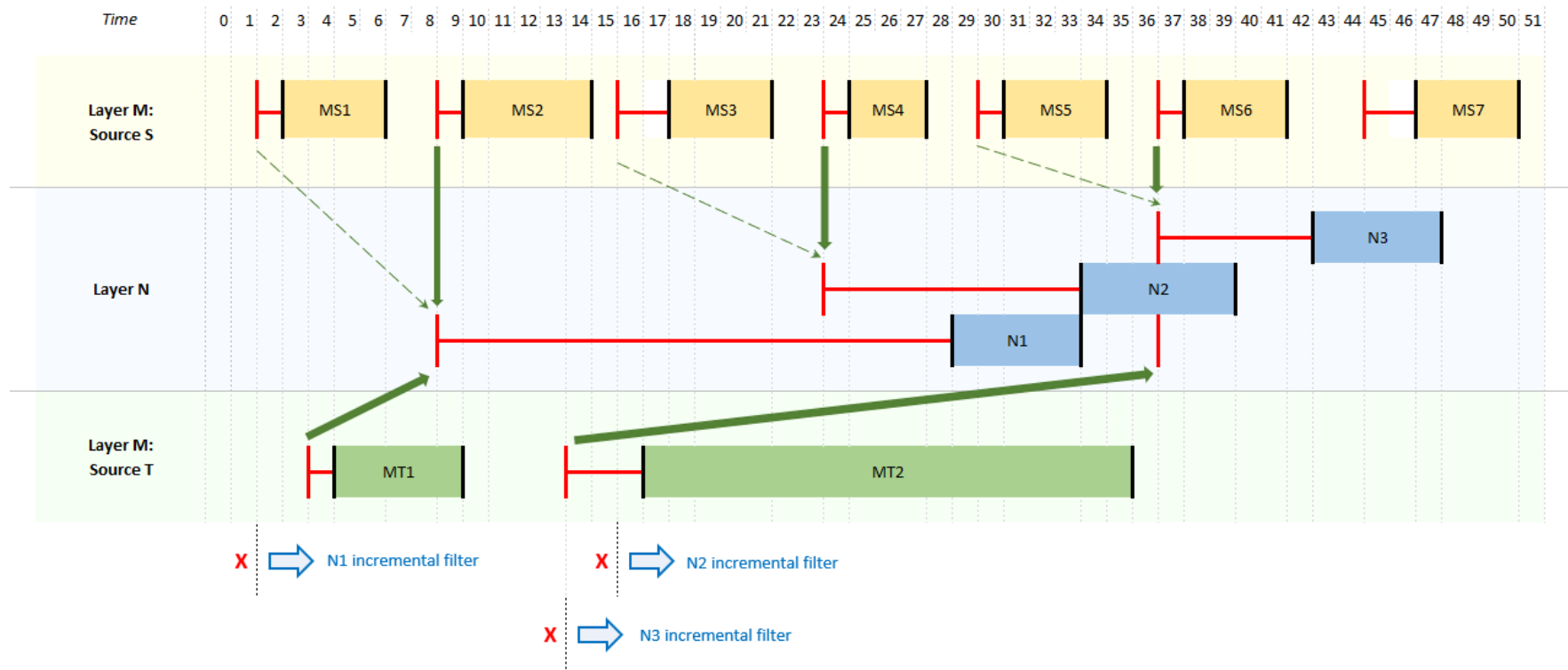
NOTES

Load MT2 has been significantly delayed (e.g. could be a catch-up run at the lower layer) and so runs with a BED earlier than the latest BED already used in the higher layer i.e. in effect it changes history.

As source S has not updated in the meantime, normal rules would therefore have this pulled into the next layer in load N2 with a BED that's already been used by load N1, but the same BED can never be used twice (would cause Type 2 merge problems/lost data). So instead a small increment is added to the BED of N2 to differentiate.

BED - Multiple Source Load, Scenario 3

Output layer loaded using historical slices of data following a period without loads, to catch-up in chunks rather than lumping all change into a small window



Load Metadata

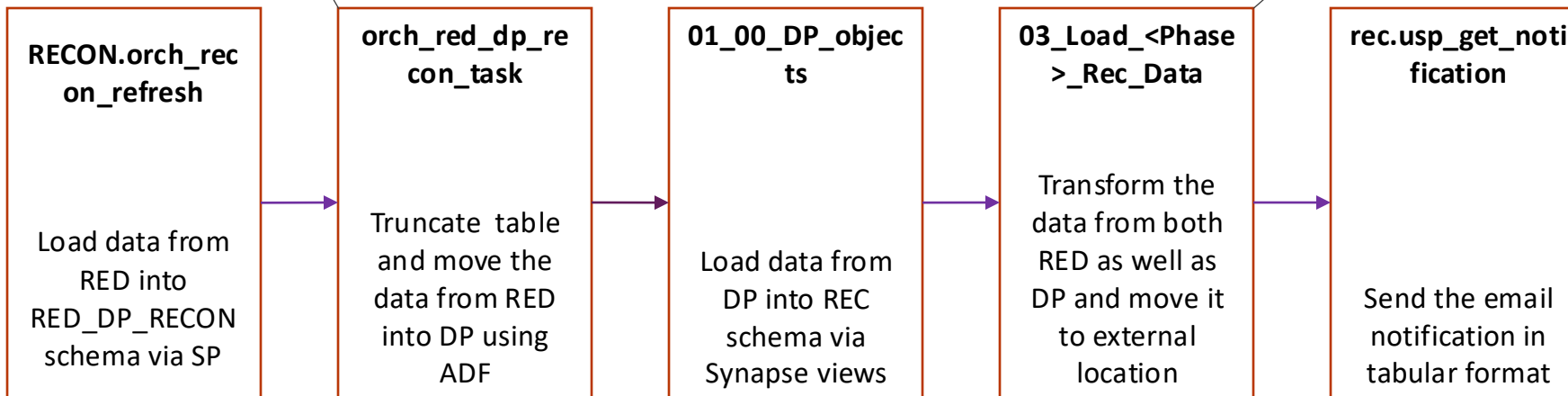
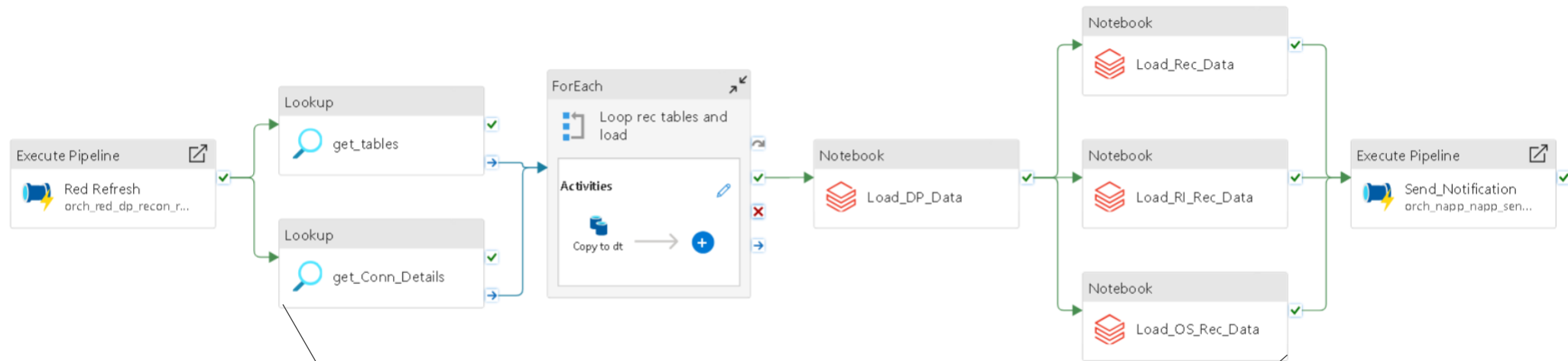
Load	Input Info			Load Config		
	Source	Input Loads	BEDs	Input BED	Incremental Filter	Output BED
N1	S	MS1, MS2	1, 8	8	1	8
	T	MT1	3	3		
N2	S	MS3, MS4	15	23	15	23
	T	-	-	3		
N3	S	MS5, MS6	29, 36	36	13	36
	T	MT2	13	13		

NOTES

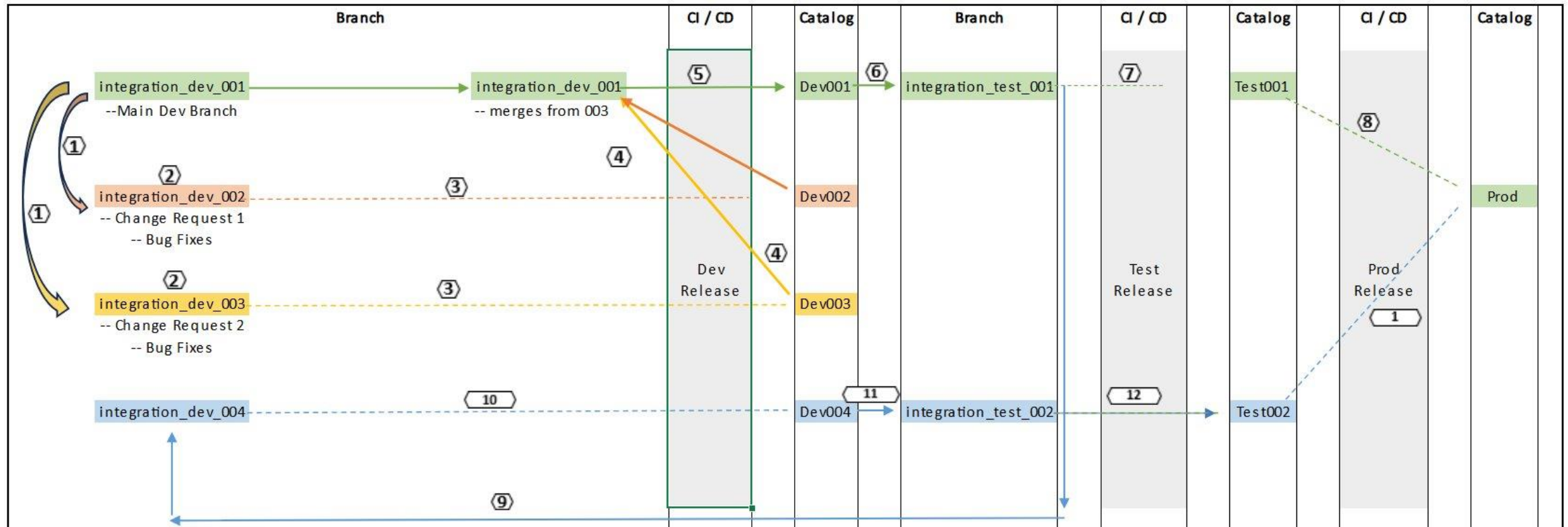
Load N1 could have potentially pulled data from source loads M1, M2, M3 and M4 all together, as these had all completed prior to start of N1. However to avoid to large a quantum of change in a single layer N load, Input BED/From and To are chosen such that N1 only targets M1 and M2.

Source T has not updated in between higher-layer loads N1 and N2, so its Input BED is unchanged.

Reconciliations



Catalogs and Squads



Please refer to below Visio for better understanding of Catalogs and squads - [Approach-Squad_Development&Release.vsd](#)

Catalogs and Squads

Step No	Step Description
1	Copy the latest code from dev_001 branch
2	Make changes to respective branch
3	Test code and Release into respective dev_001 catalog / sub environment
4	Move the changes to dev_001 branch
5	Release code into dev_001 catalog/sub environment
6	Push the code from dev_001 to main test_001 branch
7	Release into respective test_001 catalog / sub environment
8	Release into respective prod catalog / sub environment
9	Creating copy of prod for any hot fix from test_001 into dev_004 branch
10	Test code and Release into respective dev_004 catalog / sub environment
11	Move the changes to test_002 branch
12	Release code into test_002 catalog/sub environment
13	Release hot fix into respective prod catalog / sub environment

Important Links

- [Legacy-RED&GraftToCloud.xlsx](#)
- [Approach-Squad_Development&Release.vsd](#)
- [Approach-CI_CD_Overview.docx](#)
- [Approach-Handling_Deleted_Attributes.xlsx](#)
- [Approach-Data_Filtering.xlsx](#)
- [Approach-NULL_Handling_Defaults.xlsx](#)
- [Approach-BED_Specific_Fact_Load.xlsx](#)
- [Approach-Build and Data Load](#)

Thank You